

Image Processing Homework Report

Student ID: 113950011 Name: 鄭名翔

Q1 — Histogram Equalization

Method

Histogram equalization redistributes pixel intensities to achieve a more uniform distribution across the full grayscale range [0, 255], thereby enhancing the overall contrast of the image. The implementation follows these steps: (1) Compute the histogram $h(k)$ of the input image using pure NumPy. (2) Derive the cumulative distribution function $CDF(k) = \text{sum of } h(i) \text{ for } i \text{ from } 0 \text{ to } k$. (3) Build a mapping function $s(k) = \text{round}((CDF(k) - CDF_{\min}) / (N - CDF_{\min}) \times 255)$, where N is the total number of pixels and $CDF_{\min}$ is the smallest nonzero CDF value. (4) Apply the mapping via NumPy array indexing: $\text{output} = s[\text{input}]$.

Result

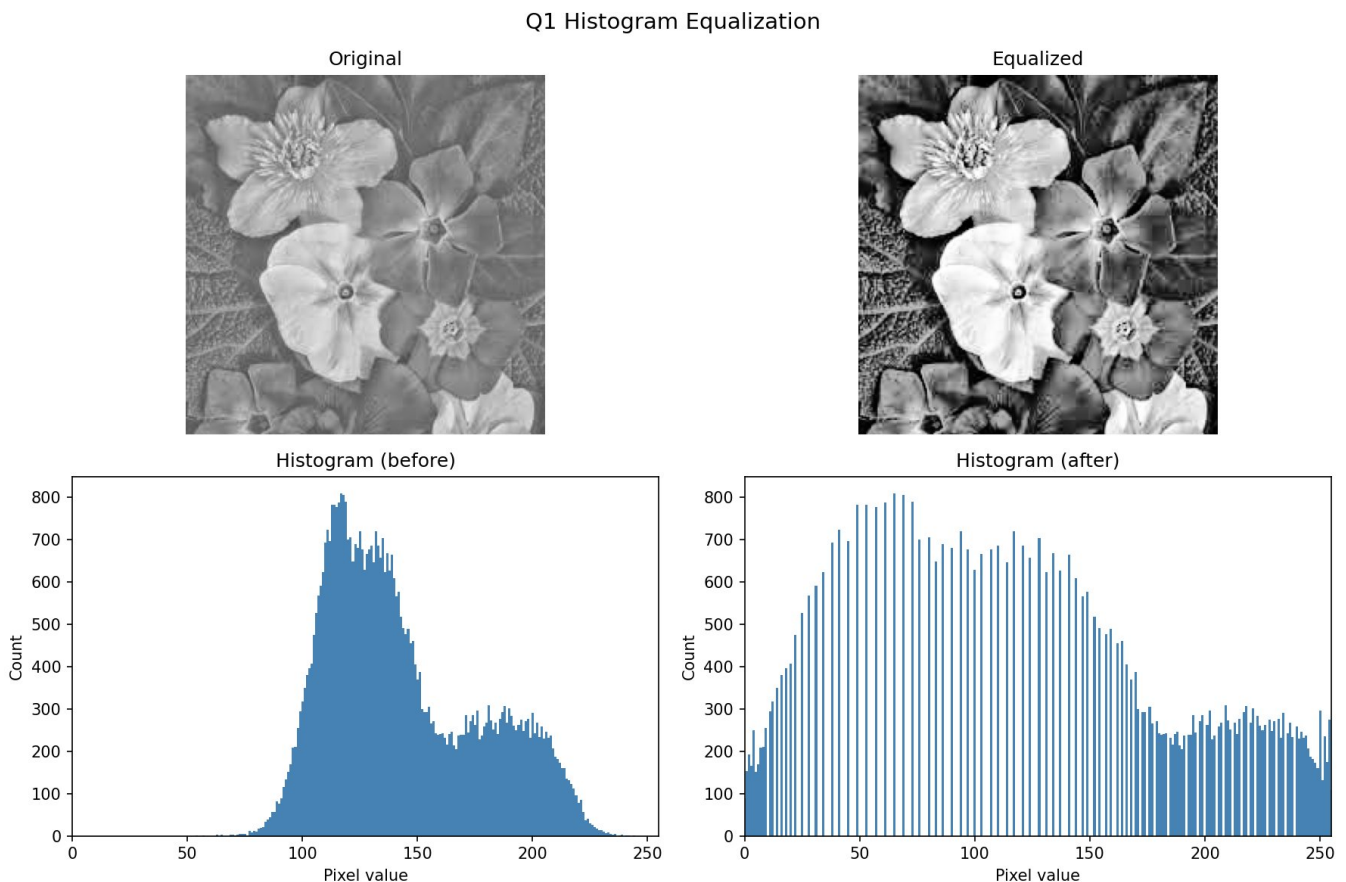


Fig. 1 — Original image (top-left) and equalized image (top-right), with histograms before and after equalization (bottom row).

The original histogram is narrowly concentrated around mid-tones (~100-140), indicating an overall low-contrast, grey appearance. After equalization the distribution spreads across the full range, producing deeper shadows and brighter highlights with noticeably improved detail in both the flowers and the leaves.

Q2 — Histogram Specification (Matching)

Method

Histogram specification transfers the tonal distribution of a reference image onto a source image, commonly used for style transfer or tonal matching. The implementation follows these steps: (1) Compute normalized CDFs for both the source and reference images (each divided by total pixel count so values lie in [0, 1]). (2) For every source gray level k , use `np.interp` to find the reference gray level j whose $CDF_{\text{ref}}(j)$ is closest to $CDF_{\text{src}}(k)$, forming a mapping table. (3) Apply the mapping table to the source image via array indexing to produce the output.

CDF & Mapping Curve

Q2 Histogram Specification - CDF & Mapping

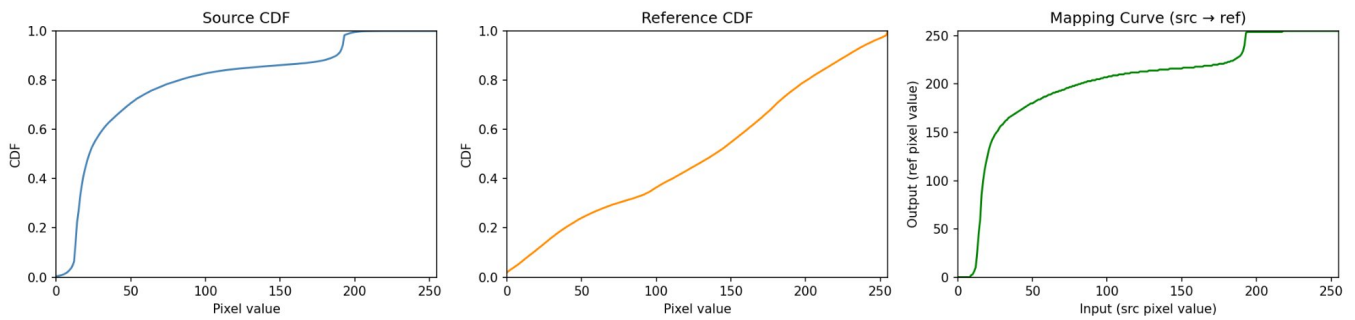


Fig. 2 — Source CDF (left), Reference CDF (centre), and Mapping Curve (right). The source CDF rises steeply near zero (dark-heavy image); the reference CDF is near-linear (broadly spread landscape). The mapping curve consequently pushes most source values toward the higher output range.

Result Images & Histograms

Q2 Histogram Specification - Images

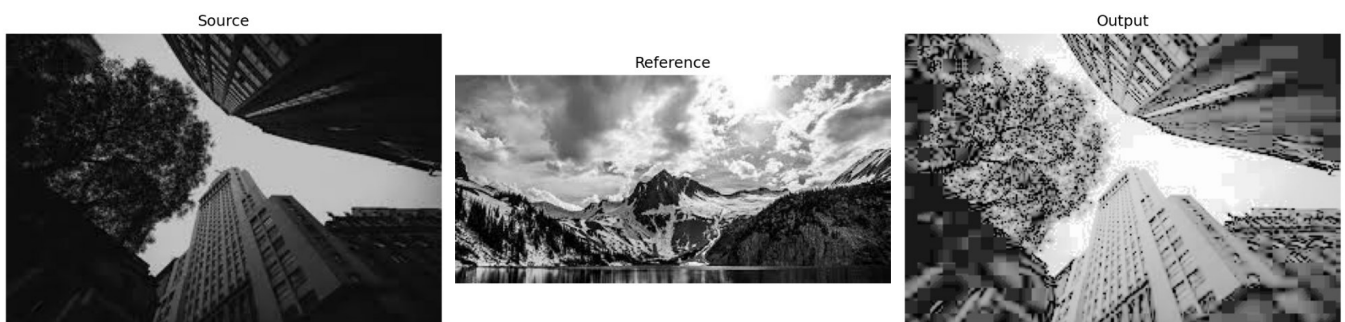


Fig. 3 — Source image (left), reference image (centre), and output image (right).

Q2 Histogram Specification - Histograms

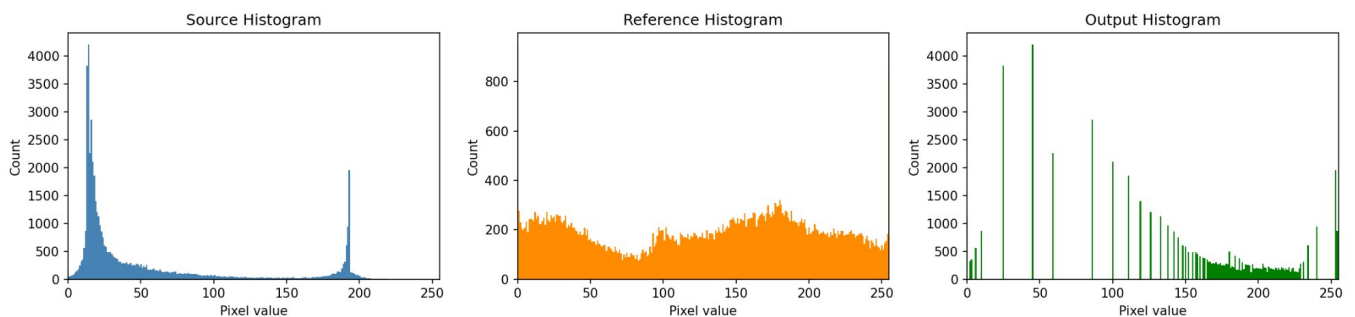


Fig. 4 — Histograms of source (left), reference (centre), and output (right). The output histogram approximately follows the broad, uniform shape of the reference, confirming that the matching procedure is working correctly.

Q3 — Spatial Filtering: Smoothing

Method

Four filter combinations are applied: Mean filter and Median filter, each with 3x3 and 5x5 kernels. Both are implemented in pure NumPy using `np.pad` (edge-replication mode) and a sliding-window loop. Mean filter: replaces each pixel with the arithmetic mean of its neighbourhood — equivalent to convolution with a uniform kernel. It reduces Gaussian noise but blurs edges. Median filter: replaces each pixel with the median value of its neighbourhood — a non-linear operation that is robust to outliers (e.g., salt-and-pepper noise) and preserves edge detail far better than the mean filter.

Result

Q3 Spatial Filtering Comparison



Fig. 5 — Original and four filtered results: Mean 3x3, Mean 5x5, Median 3x3, Median 5x5.

Q3 Mean vs Median – Difference

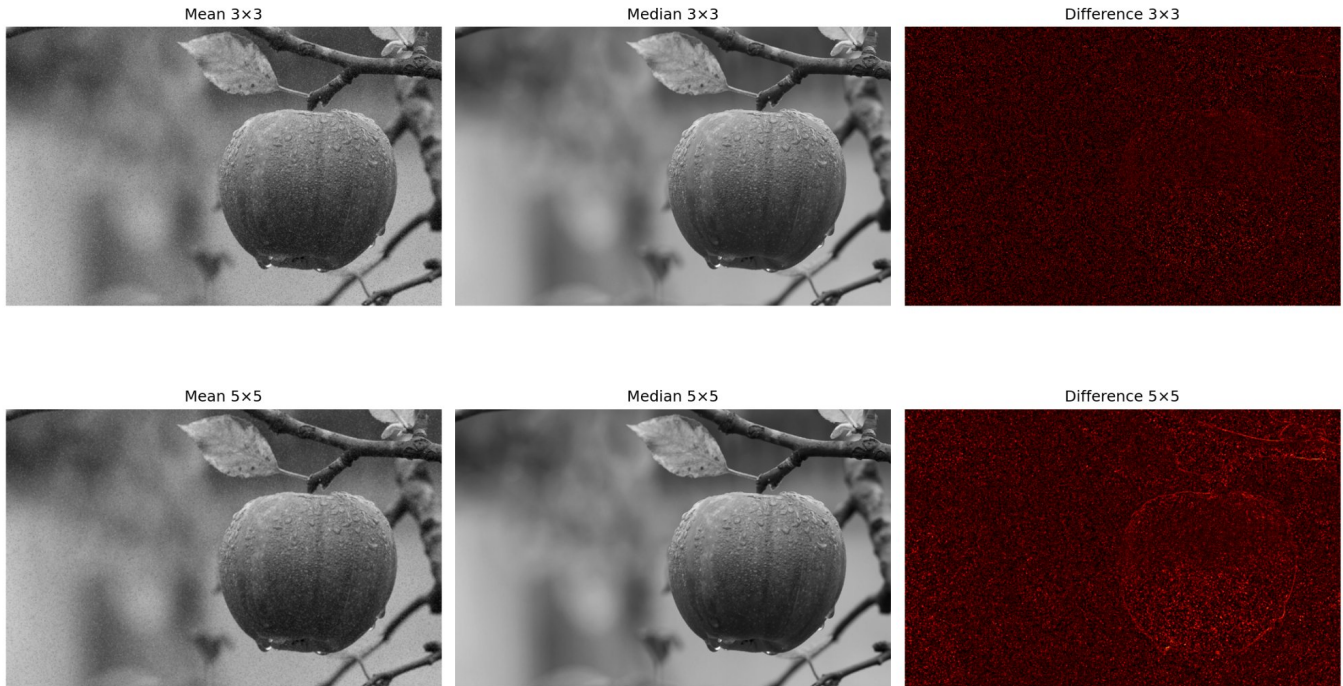


Fig. 6 — Pixel-level absolute difference between Mean and Median outputs (hot colormap; brighter = larger difference). Differences concentrate along edges and fine texture regions.

Comparison & Analysis

Both filter types reduce noise, but they differ significantly in behaviour. The Mean filter averages all pixels in the kernel equally, so sharp edges become smooth gradients and fine details are softened. With a 5x5 kernel the blurring is clearly visible in the background, where fine details largely disappear. The Median filter selects the median value, making it robust to outliers. It removes grain-like noise while preserving edge sharpness. Comparing the 5x5 results, the median version retains crisp branch contours whereas the mean version blends them into the background. The difference heatmap (Fig. 6) confirms that disagreement between the two methods is highest along object boundaries and textured surfaces — exactly where edge preservation matters most. Conclusion: For this image, which contains grain-like noise, the Median filter produces superior results because it suppresses noise while keeping structural detail intact.

Feedback

This assignment deepened my understanding of histogram-based image enhancement. Implementing everything from scratch with pure NumPy — without relying on OpenCV built-in functions — forced me to reason carefully about each mathematical step, especially the CDF normalisation and interp-based matching in Q2. The spatial filtering task highlighted the fundamental trade-off between noise suppression and detail preservation, and the difference heatmap provided an intuitive way to visualise where the two filter strategies diverge.